

Nama : Kemal Tangguh Aji Rajasa

NRP : 5025231263

Kelas : Distributed System 2025

Repository : <https://github.com/KemalRajasa/distributed-system/tree/main/tugas2>

Demo : [youtube](#)

1. Pendahuluan

1.1. Tugas yang Diberikan

Membangun lingkungan sistem terdistribusi dan mensimulasikan racing condition di GNS 3

1.2. Tujuan Percobaan

Percobaan dilakukan untuk mensimulasikan racing condition dan bagaimana mutual exclusion menangannya

1.3 Metode Percobaan

a. Membangun dan Mensimulasikan Lingkungan Distribusi

Mensimulasikan lingkungan terdistribusi dengan mengkonfigurasi topologi jaringan di GNS3 yang terdiri dari lima node *server* yaitu node 1 sampai node 5, lalu node logger dan node client untuk menjalankan script kvclient yang bertujuan untuk menginisialisasi *racing condition*.

b. Mensimulasikan Racing Condition dengan atau tanpa Mutex

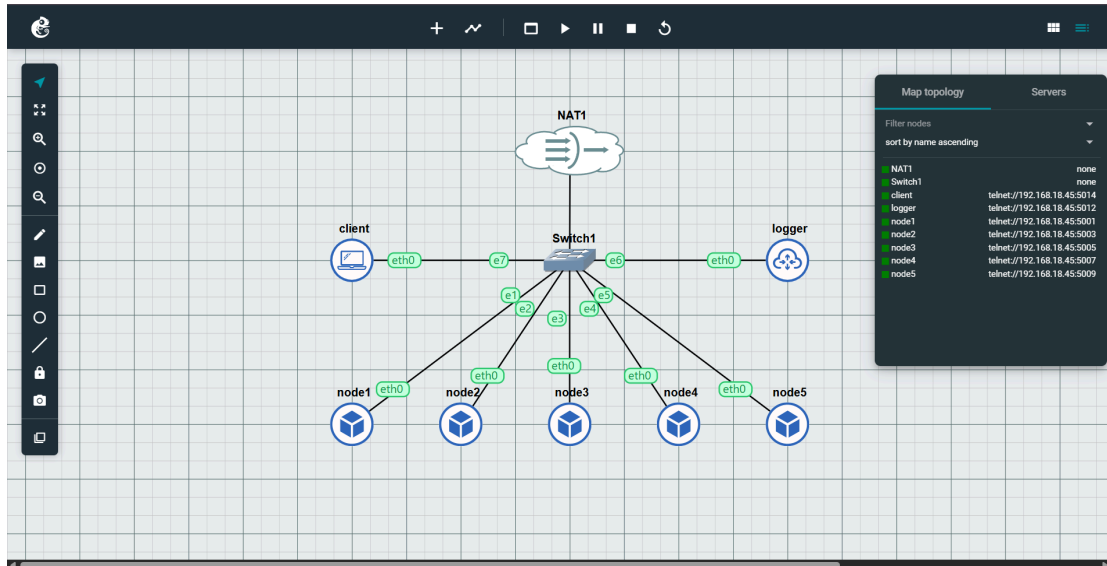
Program kv dijalankan dengan parameter enable/disable mutex, dimana ini akan memberikan output yang berbeda jika racing condition dijalankan tanpa atau dengan mutex.

c. Menganalisis Mekanisme Penanganan Racing Condition

Mengumpulkan semua lalu lintas protocol POST dan GET yang dikirim dari node client pada node *logger* lalu event POST dan GET tersebut akan dianalisa, apa dampak atau efek mutex ke racing condition, bagaimana mutex menangannya.

2. Arsitektur Percobaan

2.1 Topologi



gambar 2.1.1 screenshot topologi

Di pusatnya adalah Switch1, yang bertindak sebagai titik koneksi utama. lima node bernama node1 hingga node 5, satu node logger, dan satu node client terhubung ke switch 1. Switch1 kemudian terhubung ke NAT1 (Network Address Translation), yang menyediakan konektivitas jaringan eksternal internet untuk semua node di jaringan internal tersebut. Node 1-5 akan menjalankan program kv(dot)py yang menerima input POST atau GET dari client yang menjalankan program kvclient(dot)py lalu log event akan ditampilkan di node logger.

2.2 IP Addresses

NODE	NODE IP	VNC CONNECTION	IP:PORT
1	192.168.122.214	telnet://192.168.18.45:5001	192.168.122.39:8001
2	192.168.122.200	telnet://192.168.18.45:5003	192.168.122.189:8002
3	192.168.122.236	telnet://192.168.18.45:5005	192.168.122.7:8003
4	192.168.122.114	telnet://192.168.18.45:5007	192.168.122.159:8004
5	192.168.122.68	telnet://192.168.18.45:5009	192.168.122.159:8005
logger	192.168.122.143	telnet://192.168.18.45:5012	192.168.122.159:9000

2.3 Spesifikasi

Hardware

PART	DETAIL
CPU	Intel i5 - 12450H
GPU	Nvidia RTX 2050

RAM	16GB DDR4
OS	Windows 11 HOME

Virtual Machine

INFO	DETAIL
Software Virtualisasi	Virtual Box
Alokasi RAM	8 GB
Alokasi Core	8 Core
Node Image	royyana/netics-pc:debi-latest

Spesifikasi Node

```
root@node1:~# lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 8
On-line CPU(s) list:    0-7
Vendor ID:              GenuineIntel
Model name:             12th Gen Intel(R) Core(TM) i5-12450H
CPU family:             6
Model:                  154
Thread(s) per core:     1
Core(s) per socket:     8
Socket(s):              1
Stepping:               3
BogoMIPS:               4992.00
```

3. Implementasi dan Penggunaan

3.1. Instalasi

3.1.1 Konfigurasi Jaringan

```
auto eth0
iface eth0 inet dhcp
```

3.1.2 Instalasi Program ke Node

Node 1

```
cd root
```

```
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/node1  
ls
```

Node 2

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/node2  
ls
```

Node 3

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/node3  
ls
```

Node 4

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/node4  
ls
```

Node 5

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/node5  
ls
```

Node Logger

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/logger  
ls
```

Node Client

```
cd root  
git clone https://github.com/KemalRajasa/distributed-system/  
cd distributed-system/tugas2/src/client  
ls
```

3.2 Konfigurasi

Konfigurasi script run.sh untuk memudahkan mengeksekusi program kv.py untuk server dan konfigurasi skrip test1-3.sh yang mengeksekusi program kvclient.py untuk client.

run.sh node 1

```
#!/bin/bash

python3 ./kv.py --id 1 --tcp 8001 --udp 8101 --peers
192.168.122.200:8002=2,192.168.122.236:8003=3,192.168.122.114:8004=4,192.168.122.6
8:8005=5 --logger-addr 192.168.122.143:9000 --numnodes 5 --use-mutex 1
```

Argumen --peers digunakan untuk mengenali sesama node server, pada kasus node 1 adalah untuk mengenali node 2-5, begitu juga untuk node 2 yang harus mengenali node 1, 3, 4, dan 5. Detil konfigurasi run.sh bisa diakses pada [repository](#).

test1.sh

```
#!/bin/bash

NODES="192.168.122.214:8001,192.168.122.200:8002,192.168.122.236:8003,192.168.12
2.114:8004,192.168.122.68:8005"

# Single PUT to node 1
python3 ./kvclient.py --nodes $NODES cmd --node 1 "PUT color blue"

# GET from node 2
python3 ./kvclient.py --nodes $NODES cmd --node 2 "GET color"

# Race two writers (great for no-mutex demo)
python3 ./kvclient.py --nodes $NODES race "PUT color blue" "PUT color red"

# Read the key from ALL nodes after the race
python3 ./kvclient.py --nodes $NODES getall color
```

Skrip test1.sh bertujuan untuk menguji konsistensi node server dengan cara menginisialisasi kondisi race dimana terdapat dua node yang menjalankan operasi PUT untuk skrip test1-3.sh lengkap bisa diakses di [repository](#)

3.3 Penggunaan

Saya juga menyertakan video demo sederhana bagaimana menggunakan program, [akses video](#)

1. Instal program dengan perintah git clone.

```
root@node1:~# git clone https://github.com/KemalRajasa/distributed-system/  
Cloning into 'distributed-system'...  
remote: Enumerating objects: 221, done.  
remote: Counting objects: 100% (221/221), done.  
remote: Compressing objects: 100% (182/182), done.  
Receiving objects: 100% (221/221), 54.81 KiB | 256.00 KiB/s, done.  
remote: Total 221 (delta 56), reused 76 (delta 26), pack-reused 0 (from 0)  
Resolving deltas: 100% (56/56), done.  
root@node1:~# ls  
distributed-system  
root@node1:~# cd distributed-system/tugas2/src/  
root@node1:~/distributed-system/tugas2/src# ls  
client logger node1 node2 node3 node4 node5 program testing  
root@node1:~/distributed-system/tugas2/src#
```

gambar 3.3.1 screenshot hasil git clone

Setelah melakukan instalasi program, gunakan perintah cd untuk menuju direktori sesuai node

2. Jalankan logger pada port 9000.

```
root@logger:~# cd distributed-system/tugas2/src/logger/  
root@logger:~/distributed-system/tugas2/src/logger# bash run.bash  
bash: run.bash: No such file or directory  
root@logger:~/distributed-system/tugas2/src/logger# bash run.sh  
[LOGGER] listening on 9000  
█
```

gambar 3.3.2 screenshot logger berjalan di port 9000

Logger sudah berjalan dan akan menangkap event yang terjadi selama proses simulasi

3. Jalankan node 1-5 sebagai server (argumen --use-mutex disesuaikan 0/1).


```

===== TRACE (last 33 events) =====
-- Physical order --
t=1762741888.982983 L= 1 V=[0, 1, 0, 0, 0] node=2 MUTEX_REQ color
t=1762741889.004295 L= 1 V=[0, 1, 0, 0, 0] node=2 MUTEX_GOT color
t=1762741889.005858 L= 2 V=[0, 2, 0, 0, 0] node=2 APPLY_LOCAL color=blue
t=1762741889.007862 L= 3 V=[0, 3, 0, 0, 0] node=2 REPL_SEND color=blue
t=1762741889.011633 L= 4 V=[1, 3, 0, 0, 0] node=1 REPL_RECV color=blue
t=1762741889.025707 L= 4 V=[0, 3, 1, 0, 0] node=3 REPL_RECV color=blue
t=1762741889.028593 L= 4 V=[0, 3, 0, 1, 0] node=4 REPL_RECV color=blue
t=1762741889.029481 L= 4 V=[0, 3, 0, 0, 1] node=5 REPL_RECV color=blue
t=1762741889.031717 L= 4 V=[0, 4, 0, 0, 0] node=2 MUTEX_REL color

```

gambar 3.3.5 screenshot log event dengan pengurutan jam fisik

4. Analisis Mekanisme Komunikasi

Ada tiga mekanisme komunikasi utama yang terjadi antar node yang menjalankan script kv (server ke server).

1. Gossip (UDP)

Setiap node terus menerus menjalankan thread dan menampilkannya di terminal, tujuan dari gossip adalah untuk mendeteksi kegagalan, contohnya jika leader saat ini (misalnya node dengan id 5) dengan sengaja di-kill prosesnya dengan ctrl-c, maka node lain akan tau bahwa node 5 saat ini state nya adalah DEAD, dan akan melakukan election leader baru.

2. Replikasi Data (TCP)

Replikasi data dilakukan untuk menjaga agar data color di semua node tetap konsisten misalnya node 1 menerima perintah PUT color blue dari program kvclient, node 1 akan melakukan write color=blue, lalu akan membuka koneksi TCP ke setiap peer dan mengirim pesan untuk melakukan replikasi data bahwa color = blue

3. Distributed Mutex (TCP)

Ini bertujuan untuk menjamin hanya satu node yang boleh melakukan operasi PUT pada satu waktu untuk mencegah race condition. Program kv menggunakan algoritma koordinat terpusat pada leader. Jika node 1 ingin melakukan write color = blue, node 1 harus meminta izin untuk melakukan write ke leader, jika tidak ada node yang menggunakan resource write pada waktu yang sama, maka resource untuk write akan diberikan ke node 1 dan menolak permintaan write dari node lain pada waktu tersebut.

5. Simulasi Race Condition

Simulasi dibagi menjadi dua bagian yaitu, race condition tanpa mutex dan race condition dengan mutex

1. Racing Condition Tanpa Mutex

Skip kv.py pada node server dieksekusi dengan parameter --use-mutex 0.


```

root@client:~/distributed-system/tugas2/src/client# bash test1.sh
[192.168.122.200:8002] PUT color blue -> OK (75.15 ms)
[192.168.122.236:8003] GET color -> blue (28.45 ms)
[192.168.122.214:8001] PUT color blue -> OK (132.39 ms)
[192.168.122.200:8002] PUT color red -> OK (320.93 ms)
[192.168.122.214:8001] GET color -> red (18.82 ms)
[192.168.122.200:8002] GET color -> blue (34.95 ms)
[192.168.122.236:8003] GET color -> red (20.77 ms)

```

gambar 5.2.1 hasil bash test1.sh tanpa mutex

Berdasarkan gambar 5.2.1 pada baris ke-3 dan 4, operasi PUT secara bersamaan yang diinisialisasi oleh perintah race dalam file test1 diterima oleh node 1 dan 2, dimana ini menyebabkan node 1 dan 2 mendapat akses untuk melakukan write secara bersamaan menghasilkan inkonsistensi data, hasil lengkap bisa dilihat pada link file [log.txt](#)

2. Racing Condition Ditangani Mutex

Skip kv.py pada node server dieksekusi dengan parameter --use-mutex 1.

```

root@client:~/distributed-system/tugas2/src/client# bash test1.sh
[192.168.122.200:8002] PUT color blue -> OK (54.09 ms)
[192.168.122.236:8003] GET color -> blue (10.33 ms)
[192.168.122.214:8001] PUT color blue -> ERR: timed out (0.00 ms)
[192.168.122.200:8002] PUT color red -> OK (359.37 ms)
[192.168.122.214:8001] GET color -> red (31.99 ms)
[192.168.122.200:8002] GET color -> red (68.92 ms)
[192.168.122.236:8003] GET color -> red (117.78 ms)

```

gambar 5.2.2 hasil bash test1.sh

Berdasarkan gambar 5.2.2, hanya satu operasi PUT yang diterima pada eksekusi ke-3 file test1.sh (eksekusi dengan argumen race) yang melakukan operasi PUT bersamaan menjadi indikasi cara mutex menangani race condition yaitu dengan menolak salah satu operasi PUT.

```

-- Lamport order --
L= 1 t=1762744679.113239 V=[0, 1, 0, 0, 0] node=2 MUTEX_REQ color
L= 1 t=1762744679.124752 V=[0, 1, 0, 0, 0] node=2 MUTEX_GOT color
L= 2 t=1762744679.125942 V=[0, 2, 0, 0, 0] node=2 APPLY_LOCAL color=blue
L= 3 t=1762744679.127889 V=[0, 3, 0, 0, 0] node=2 REPL_SEND color=blue
L= 4 t=1762744679.149324 V=[1, 3, 0, 0, 0] node=1 REPL_RECV color=blue
L= 4 t=1762744679.154182 V=[0, 4, 0, 0, 0] node=2 MUTEX_REL color
L= 4 t=1762744679.147773 V=[0, 3, 1, 0, 0] node=3 REPL_RECV color=blue
L= 5 t=1762744679.373616 V=[2, 3, 0, 0, 0] node=1 MUTEX_REQ color
L= 5 t=1762744679.372782 V=[0, 5, 0, 0, 0] node=2 MUTEX_REQ color
L= 5 t=1762744679.493548 V=[0, 5, 0, 0, 0] node=2 MUTEX_GOT color
L= 5 t=1762744679.283341 V=[0, 3, 2, 0, 0] node=3 GET color
L= 6 t=1762744679.535712 V=[0, 6, 0, 0, 0] node=2 APPLY_LOCAL color=red
L= 7 t=1762744679.540301 V=[0, 7, 0, 0, 0] node=2 REPL_SEND color=red
L= 8 t=1762744679.698243 V=[3, 7, 0, 0, 0] node=1 REPL_RECV color=red

```

```

L= 8 t=1762744679.722684 V=[0, 8, 0, 0, 0] node=2 MUTEX_REL color
L= 8 t=1762744679.719082 V=[0, 7, 3, 0, 0] node=3 REPL_RECV color=red
L= 9 t=1762744681.552046 V=[4, 7, 0, 0, 0] node=1 GET color
L= 9 t=1762744688.397316 V=[4, 7, 0, 0, 0] node=1 MUTEX_GOT color
L= 9 t=1762744681.610819 V=[0, 9, 0, 0, 0] node=2 GET color
L= 9 t=1762744681.692608 V=[0, 7, 4, 0, 0] node=3 GET color
L= 10 t=1762744688.406233 V=[5, 7, 0, 0, 0] node=1 APPLY_LOCAL color=blue
L= 11 t=1762744688.408532 V=[6, 7, 0, 0, 0] node=1 REPL_SEND color=blue
L= 12 t=1762744688.429389 V=[7, 7, 0, 0, 0] node=1 MUTEX_REL color
L= 12 t=1762744688.418821 V=[6, 10, 0, 0, 0] node=2 REPL_RECV color=blue
L= 12 t=1762744688.427352 V=[6, 7, 5, 0, 0] node=3 REPL_RECV color=blue

```

tabel 5.2.2 log lampport order test1.sh

Berdasarkan output log lampport order data color tetap konsisten color = blue, hasil log lengkap bisa diakses pada link [log.txt](#)

3. Simulasi 5 Node dengan Mutex

```

root@client:~/distributed-system/tugas2/src/client# bash test1.sh
[192.168.122.200:8002] PUT color blue -> OK (313.15 ms)
[192.168.122.236:8003] GET color -> blue (23.76 ms)
[192.168.122.214:8001] PUT color blue -> OK (361.43 ms)
[192.168.122.200:8002] PUT color red -> ERR: timed out (0.00 ms)
[192.168.122.214:8001] GET color -> blue (8.17 ms)
[192.168.122.200:8002] GET color -> blue (36.91 ms)
[192.168.122.236:8003] GET color -> blue (10.70 ms)
[192.168.122.114:8004] GET color -> blue (41.73 ms)
[192.168.122.68:8005] GET color -> blue (11.93 ms)

```

gambar 5.2.3 screenshot hasil bash test1.sh

Sama seperti simulasi dengan mutex sebelumnya yang hanya menggunakan 3 node, meski node ditambah menjadi lima, hanya satu operasi PUT yang diterima, hasil log bisa diakses pada link file [log.txt](#)

6. Kesimpulan

Mutual exclusion dibutuhkan untuk menjaga konsistensi data dan sinkronisasi, dibuktikan pada percobaan simulasi tanpa mutex, masing-masing node memberikan respons yang berbeda ketika client menanyakan color dengan perintah GET. Mekanisme mutual exclusion untuk menjaga konsistensi adalah dengan node yang ingin melakukan operasi write harus meminta izin ke leader dengan metode MUTUAL_REQ dan leader hanya mengizinkan satu node yang memiliki resource untuk write dalam satu waktu.